



DOCUMENT INTERNE

Documentation technique Lomdie

Architecture, exploitation et reprise du site public et du back-office.

À l'usage des développeurs et responsables techniques

Version mise à jour - 11 août 2026

01 Architecture actuelle

Composant	Rôle
Next.js 16.3 / React 19	Application unique App Router pour lomdie.com et admin.lomdie.com, Cache Components actif.
Supabase	Postgres, Auth et Storage. Source de vérité unique ; Airtable est abandonné.
Vercel	Projet lomdie-web dans l'équipe Lomdie, déploiement continu depuis GitHub.
Resend	Emails transactionnels et SMTP Supabase Auth.
Cal.com	Calendrier intégré et webhook signé vers /api/webhooks/cal-com.

02 Comptes et cibles officielles

- GitHub : organisation Lomdie, dépôt Lomdie/lomdie-web, branche de production main.
- Vercel : équipe Lomdie, projet lomdie-web. Domaines lomdie.com, www.lomdie.com et admin.lomdie.com.
- Supabase : organisation et projet Lomdie dédiés, distincts des comptes personnels historiques.
- Compte technique commun : `developpeur@lomdie.com`. Les secrets restent dans les coffres des services, jamais dans ce document.

Toujours vérifier l'organisation et le projet avant toute action. Ne jamais déployer Lomdie vers un espace Vercel personnel.

03 Code et déploiement

- Lire AGENTS.md et la documentation locale de Next.js 16 avant toute modification du rendu ou du cache.
- Exécuter npm run lint, npx tsc --noEmit et npm run build selon le risque du changement.
- Le push sur main déclenche la production Vercel Lomdie via l'intégration GitHub.
- Le fichier local .vercel/project.json doit pointer vers le projet Lomdie officiel ; il est exclu de Git.
- Après déploiement, contrôler le commit, l'état READY, les URL réelles et les erreurs d'exécution.

La CLI locale peut rester connectée à un compte personnel. Si elle est utilisée, imposer explicitement la bonne équipe et vérifier .vercel/project.json ; le flux GitHub reste la voie recommandée.

04 Parcours métier

Entrée	Flux de données
/candidature	Crée un candidat au statut nouvelle_candidature. Visible dans Prospects, jamais dans Candidatures détaillées.
Réservation découverte	Cal.com envoie le webhook ; la réservation est rapprochée du prospect par email.
/prendre-rendez-vous	Après paiement : envoi du dossier complet, photos Storage, changement de statut, puis affichage du calendrier.
Mise en relation	candidate_matches relie deux candidats et conserve le statut bidirectionnel.
Newsletter homepage	Insère une adresse unique dans newsletter_subscribers ; consultation admin uniquement.

05 Données principales

Table	Responsabilité
candidates	Prospects et dossiers détaillés, statut, profil, critères et visibilité.
candidate_matches	Paires de candidats, étape de relation et notes.
calendly_bookings	Rendez-vous découverte/après paiement et liens de réunion.
site_content / pricing_plans	Textes publics et offres administrables.
testimonials / faq_items / blog_posts	Contenus éditoriaux structurés.
team_members	Équipe publique.
contact_submissions / newsletter_subscribers	Messages de contact et emails collectés.
integration_secrets	Secret du webhook Cal.com côté serveur uniquement.

06 Stockage et fichiers

Bucket	Accès
team-photos	Lecture publique, écriture admin.
blog-assets	Lecture publique, écriture admin.
candidate-photos	Photos privées des dossiers, URLs signées à durée limitée dans l'admin.

Ne jamais rendre candidate-photos public. Les chemins relatifs sont conservés en base et résolus côté serveur pour l'admin.

07 Authentification et sécurité

- Supabase Auth protège /admin via la session serveur.
- Tout utilisateur Auth actif dispose actuellement d'un accès admin complet ; il n'existe pas encore de rôles fins.
- Les Server Actions admin doivent vérifier l'utilisateur et respecter les politiques RLS.
- La service role key reste exclusivement côté serveur pour les formulaires publics et opérations privilégiées.
- Les données de religion et de tribu exigent une attention RGPD et la traçabilité du consentement explicite.
- Le webhook Cal.com vérifie x-cal-signature-256 avec HMAC SHA-256 avant toute écriture.

08 Variables et intégrations

Variable	Rôle
NEXT_PUBLIC_SUPABASE_URL	URL du projet Supabase officiel.
NEXT_PUBLIC_SUPABASE_ANON_KEY	Clé publique Supabase.
SUPABASE_SERVICE_ROLE_KEY	Accès serveur privilégié, secret.
RESEND_API_KEY	Emails transactionnels.
NEXT_PUBLIC_SITE_URL	Origine publique utilisée dans les liens.
CAL_WEBHOOK_SECRET	Secret optionnel du webhook ; un secret chiffré fonctionnel peut aussi être lu côté serveur.

Ne jamais copier les valeurs dans Git, un ticket, une capture ou ce PDF.

09 Cache et cohérence

Les lectures publiques utilisent Cache Components, cacheLife et cacheTag. Les mutations administratives doivent invalider le tag correspondant.

Après une migration ou une correction urgente, utiliser `revalidateTag(tag, { expire: 0 })` lorsque l'API de cette version l'exige. Un redéploiement ne garantit pas à lui seul l'expiration des données en cache.

Tags courants : team-members, site-content, pricing-plans, testimonials, faq-items, blog-posts et public-profiles.

10 Historique Airtable

Airtable a servi de source historique pour l'import initial des adhérents. Les données ont été migrées vers Supabase, y compris les photos vers candidate-photos.

Airtable n'est plus une dépendance applicative, aucune synchronisation continue n'est attendue et le script ponctuel d'import a été retiré du dépôt. Les migrations SQL historiques restent conservées : elles décrivent le schéma réellement appliqué et ne doivent pas être supprimées.

11 Exploitation et reprise

Contrôle	Attendu
Git	Arbre propre hors changements utilisateur ; commit ciblé sur main.
Vercel	Déploiement production READY dans l'équipe Lomdie, commit exact.
Site public	Pages principales, formulaire de candidature et newsletter accessibles.
Admin	Connexion, Prospects, Candidatures, CRUD, Newsletter et contenu accessibles.
Cal.com	Webhook reçu, ligne créée, lien de réunion exploitable.
Supabase	Migrations appliquées, RLS actives, Storage conforme.
Observabilité	Aucune erreur runtime nouvelle après déploiement.